

Scott-Toolkit v2 — How to Use It

This is your personal cheat sheet. The toolkit handles context engineering — making sure Claude Code always knows where you left off, what you're building, and what lessons you've learned. It works alongside GSD (project management) and Superpowers (dev methodology).

Starting a Session

Just open Claude Code in your project folder. The toolkit handles the rest:

- **Session-start hook fires automatically** — it checks for context files and shows you what's available
 - **If you see "Resume file found"** — Claude will read `.claude-resume.md` and know exactly where you left off
 - **If you see "No resume file"** — type `/scott:resume-project` for a guided walkthrough of getting back up to speed
- If you're in your home directory (`~`), the hook shows your active projects list. Pick one and `cd` into it.

The Slash Commands

These are your workflows. Type them and Claude walks you through each one.

Project Workflows

Command	When to use it
<code>/scott:new-project</code>	Starting something from scratch
<code>/scott:new-feature</code>	Adding a feature to an existing project
<code>/scott:resume-project</code>	Picking up a project after time away
<code>/scott:retro</code>	After finishing a milestone — capture lessons
<code>/scott:handoff</code>	When a prototype is ready for Gary

Learning & Improvement

Command	When to use it
<code>/scott:log-success</code>	Something went really well — capture it
<code>/scott:log-error</code>	Something went wrong — capture it
<code>/scott:source-review</code>	Compare context engineering sources against your toolkit
<code>/scott:toolkit-update</code>	When you want to improve the toolkit itself
Toolkit Spa Day	Monthly: consolidate rules/skills, review instinct candidates, remove contradictions

Reference & Knowledge

Command	When to use it
<code>/scott:debug</code>	Structured 5-phase debugging workflow
<code>/scott:surrealdb</code>	SurrealDB syntax, schema patterns, query optimization
<code>/scott:n8n-reference</code>	Comprehensive n8n automation reference
<code>/scott:automation-guide</code>	Best practices for reliable automations
<code>/scott:save-tweet</code>	Extract tweet/thread content into source files

Tools & Utilities

Command	When to use it
<code>/scott:remind</code>	Send a Telegram reminder to Scott or Brett
<code>/scott:sync</code>	Sync config and code between machines
<code>/scott:pdf</code>	Recreate a PDF in HTML/CSS, convert to

Business Context (Advosy)

| Command | When to use it | |-----|-----| | `/advosy:context` | Company structure, leadership, subsidiaries, contacts | | `/advosy:claimsforce` | Claimsforce (EspoCRM) workflows, webhooks, n8n patterns | | `/advosy:crm` | Advosy CRM design system, layout patterns, mockup-building | You don't need to memorize these. Just describe what you want to do and Claude will suggest the right one.

What Happens Automatically (You Don't Need to Do Anything)

When you start a session

- Hook scans for project context files (CLAUDE.md, PRD.md, todo.md, resume file)
- Hook rebuilds `~/Sites/Global/ACTIVE-PROJECTS.md` from all your project resume files
- You see a short summary of what's available

When context is about to compact

- Hook saves a mechanical backup (`.context-snapshot.md`)
- Hook tells Claude to write a proper resume file (`.claude-resume.md`)
- Hook prompts Claude to capture any notable session patterns to `~/claude/instinct-candidates.md`
- **You might see Claude write a file right before compaction — that's normal and intentional**
- **After compaction**, Claude re-reads the resume file, snapshot, current task, and any offloaded files before continuing

When a session ends

- Hook reminds Claude to write the resume file and update docs
- Hook prompts Claude to capture any notable session patterns to `~/claude/instinct-candidates.md`
- This is your safety net — even if you forget, the reminder fires

Guard hooks (always running)

- **git push blocked** — prevents accidental pushes (use `/scott:bypass` or confirm manually)
 - **Destructive commands blocked** — `rm -rf`, `git reset --hard`, etc. require explanation
 - **CLAUDE.md protected** — can't be overwritten without confirmation
 - **npm install blocked** — new dependencies need your approval
-

Pausing Work (The Most Important Part)

When you're done for the day or switching projects, you have two options:

Option A: Just close Claude Code

The session-end hook will remind Claude to write the resume file. If Claude has time before the session closes, it writes `.claude-resume.md` with everything needed to continue.

Option B: Tell Claude "let's stop here"

Claude will:

- Write/update `.claude-resume.md` with current state
- Update CLAUDE.md's Current Status section
- Mark completed tasks in `tasks/todo.md`

Option B is better because Claude has full context when you explicitly pause. If you just close the window, the hook fires but Claude might not have time to respond.

If context fills up mid-session

The PreCompact hook fires automatically. Claude writes the resume file before compaction compresses the conversation. After compaction, Claude reads the resume file back and continues. You shouldn't notice any disruption.

The Resume File (`.claude-resume.md`)

This is the magic. It's a ~10-line file in your project root that tells the next session exactly where to pick up. **You never write this file.** Claude writes it. It contains:

- What workflow/phase you were in
- What's done and what's next
- Key decisions made during the session

You never read this file manually either. Claude reads it at session start. It's a machine-to-machine handoff note. If you're curious, it looks like this: ```

Resume Context

Workflow: /scott:new-feature — Phase 4 (Build) PM Mode: GSD Branch: feat/user-auth Done: Mini-PRD approved.
Schema updated. Current: Auth middleware written, not connected to routes. Next: Connect middleware, then verify.
Decisions: JWT over sessions. 15min access / 7day refresh. ```

Active Projects List (`~/Sites/Global/ACTIVE-PROJECTS.md`)

This file is auto-generated every time you start a session. It lists all projects that have a `.claude-resume.md` file. You can glance at it to see what's in flight: ```

Project	Last Worked	Status	Path
life-os	2026-03-01	Phase 3: data model	~/Sites/life-os
advosy-crm	2026-02-28	Feature: user auth	~/Sites/advosy-crm

Don't edit this file. It gets rebuilt from resume files every session. When a project is done and its resume file is removed, it disappears from this list automatically.

Multi-Machine Sync

The toolkit lives at `~/Sites/Global/scott-toolkit/` and is a git repo. To sync across machines: ``` bash

On any machine — pull latest

cd ~/Sites/Global/scott-toolkit && git pull

If hooks/rules/skills seem off — re-deploy

./setup.sh

On Brett's machine (different path)

./setup.sh --toolkit-path /path/to/scott-toolkit ` The setup script uses symlinks, so after `git pull` most changes take effect immediately without re-running setup. Re-run setup only if new hooks or skills were added.

Quick Reference: What Lives Where

| Location | What's there | Who maintains it | |-----|-----|-----| | ~/Sites/Global/scott-toolkit/ | The toolkit repo (source of truth) | You + Claude via /scott:toolkit-update || ~/.claude/hooks/ | Deployed hooks (symlinked to toolkit) | setup.sh || ~/.claude/rules/ | Behavior rules (symlinked to toolkit) | setup.sh || ~/.claude/skills/scott-*/ | Deployed workflow skills | setup.sh || ~/.claude/settings.json | Hook registrations, plugins | Manual (machine-specific) || /.claude-resume.md | Resume file for that project | Claude (automatically) || ~/Sites/Global/ACTIVE-PROJECTS.md | Master project list | Session-start hook (automatically) |

What Runs Without You Thinking About It

Everything below happens behind the scenes. You'll never need to trigger, configure, or maintain any of it. It's listed here so you know what's going on if you see something unexpected.

Files that write themselves

| File | What it is | When it's created/updated | By whom | |-----|-----|-----|-----| | .claude-resume.md | Per-project handoff note (~10 lines) | Before compaction, at session end, at workflow end | Claude || .context-snapshot.md | Mechanical backup (git state, todos) | Right before compaction | Hook (shell script) || ~/Sites/Global/ACTIVE-PROJECTS.md | Master list of all in-flight projects | Every session start | Hook (shell script) || .claude/tool-output-overflow/*.md | Offloaded large tool results (>4KB) | When a tool returns a large result | Hook (shell script) || ~/.claude/instinct-candidates.md | Auto-captured session patterns for review | Before compaction + session end | Claude (prompted by hook) | You never create, edit, or delete these files. They're machine-to-machine communication.

Hooks that fire on their own

| Hook | When it fires | What it does | What you see | |-----|-----|-----|-----| | session-start.sh | Every time you open Claude Code | Scans for resume files, rebuilds active projects list, tells Claude what context exists | A 2-3 line message at the top of your session || pre-compact.sh | When the context window is about to compress | Saves mechanical backup, tells Claude to write the resume file immediately | Claude suddenly writing a file — that's normal || session-end.sh | When Claude Code is closing | Reminds Claude to write resume file + update docs | A short checklist reminder || guard-git-push.sh | Every time Claude tries to git push | Blocks it until you confirm | "Git push blocked" message || guard-destructive.sh | Every time Claude tries rm -rf , git reset --hard , etc. | Blocks it and explains why | "Blocked: [command] would..." message || guard-claude-md.sh | Every time Claude tries to edit CLAUDE.md or MEMORY.md | Blocks it until you confirm | "CLAUDE.md modification blocked" message || guard-npm-

install.sh | Every time Claude tries to install packages | Blocks it and lists the packages | "npm install blocked — packages: ..." message | | **offload-large-output.sh** | After every tool use | Writes tool results >4KB to `.claude/tool-output-overflow/` to prevent context bloat | "Large tool output saved to..." message | | **extract-instincts.sh** | Before compaction + session end | Prompts Claude to note session patterns to `~/ .claude/instinct-candidates.md` | Claude may write a quick pattern note | | **pre-completion-checklist.sh** | When a session ends | Checks for uncommitted changes, stale todo.md, missing resume file, and stale lessons.md. Strong visual warning but doesn't block. | A bordered checklist with item count | | **auto-format.sh** | After every Write or Edit (.js/.ts/.vue/.css/.json) | Runs Prettier if the project has it configured. Uses local binary for speed. Notifies Claude when formatting changes occur. | "Auto-formatted [file] with Prettier" (or nothing if no Prettier) | | **context-reminders.sh** | After every tool use | Tracks session duration and tool count via /tmp state file. Warns at 60 min and 100 tool uses. Replaces behavioral self-monitoring. | "Context Health Check" box with suggestions | | **bash logger** | Every Bash command Claude runs | Logs the command to `~/ .claude/bash-commands.log` | Nothing — completely silent | | **GSD context monitor** | After every tool use | Tracks how full the context window is | Warning when context reaches 80%+ | | **GSD update checker** | At session start | Checks if GSD has updates available | Update notification if one exists | | **GSD status line** | Continuously | Shows project/phase info in the status bar | A status line at the bottom of your terminal | | **Notification sound** | When Claude needs your attention | Plays a sound + macOS notification | "Blow" sound + popup |

Behavior rules always loaded

These rules live in `~/ .claude/rules/` and Claude reads them automatically in every session, every project. You never need to reference them. | Rule | What it tells Claude | |-----|-----| | **claude-behavior.md** | Use Superpowers for dev methodology, GSD for project management, toolkit for learning capture. Enter plan mode for complex tasks. Pre-completion verification gate (tests pass, git clean, todo updated, feature works). Task contracts (define completion criteria upfront with immutable tests). Doom-loop detection (3+ edits = re-plan or fresh subagent with contract). Subagent trigger (3+ files = spawn subagent). Subagent iterative retrieval (evaluate results, follow up if incomplete, max 3 cycles). Named agent roles with restricted tool sets (researcher, planner, reviewer, executor). Pattern referencing (point to existing code patterns instead of describing verbally). Checkpoint commits before significant refactors. Context rot monitored by context-reminders hook (60min + 100 tool uses). Post-compaction recovery (re-read resume, snapshot, task, offloaded files). Neutral prompting (avoid sycophancy bias in investigations). | | **code-style.md** | TypeScript strict mode, Vue 3 Composition API, Tailwind v4, Pinia, 2-space indent, single quotes. | | **n8n-sync.md** | Keep local tools-needing-setup file and n8n reminder workflow in sync. |

Symlinks (why updates "just work")

The hooks and rules in `~/ .claude/` are symlinks — shortcuts that point back to `~/Sites/Global/scott-toolkit/`. When you `git pull` toolkit updates, the symlinks still point to the same files, so the updates take effect instantly. No re-deploy needed unless brand new files were added.

Native Claude Code Commands

These are built-in — no setup needed. Just type them.

Session Management

| Command | What it does | |-----|-----| | `/clear` | Clear conversation and free context | | `/compact [instructions]` | Compress conversation (optional focus instructions) | | `/resume` | Resume a previous session (interactive picker) | | `/fork [name]` | Fork conversation at this point — try two approaches safely | | `/rename [name]` | Name current session for easy finding later | | `/export [filename]` | Save conversation as plain text | | `/rewind` | Rewind code and/or conversation to a checkpoint (`Esc+Esc` shortcut) |

Code & Git

| Command | What it does | |-----|-----| | /diff | Interactive diff viewer for uncommitted changes | | /review | Review a PR for quality, correctness, security, tests | | /security-review | Scan pending changes for security vulnerabilities | | /pr-comments [PR] | Fetch and display comments from a GitHub PR | | /plan | Enter plan mode (read-only analysis, then propose changes) |

Model & Output

| Command | What it does | |-----|-----| | /model [model] | Change AI model (sonnet, opus, haiku) | | /fast [on|off] | Toggle fast mode (same Opus model, faster output) | | /output-style [style] | Switch between Default, Explanatory, Learning styles |

Configuration

| Command | What it does | |-----|-----| | /config | Open settings interface | | /permissions | View or update tool permissions | | /memory | Edit CLAUDE.md files and auto-memory | | /hooks | Manage hook configurations | | /plugin | Manage plugins | | /keybindings | Configure keyboard shortcuts | | /statusline | Configure the status line display | | /theme | Change color theme | | /vim | Toggle vim editing mode |

Tools & Integration

| Command | What it does | |-----|-----| | /mcp | Manage MCP server connections | | /ide | Manage IDE integrations | | /chrome | Configure Chrome browser automation | | /add-dir | Add a working directory to current session | | /skills | List all available skills | | /tasks | List and manage background tasks |

Info & Diagnostics

| Command | What it does | |-----|-----| | /help | Show help and available commands | | /cost | Token usage and spending stats | | /usage | Plan limits and rate limit status | | /context | Visual grid of context window usage | | /stats | Session history, streaks, usage patterns | | /insights | Generate report analyzing your Claude Code sessions | | /doctor | Diagnose installation issues | | /release-notes | View changelog | | /status | Version, model, and account info |

Useful Keyboard Shortcuts

| Shortcut | What it does | |-----|-----| | Esc + Esc | Rewind/summarize (same as /rewind) | | Shift+Tab | Cycle permission modes (normal → plan → auto) | | Ctrl+V | Paste image from clipboard | | Ctrl+T | Toggle task list | | Ctrl+0 | Toggle verbose output (see Claude's thinking) | | Ctrl+B | Background a running task | | @ | File path autocomplete (type @ then start typing) | | ! | Bash mode (run command directly) | | ? | Show available shortcuts |

Active Features

Phase Auto-Advancement

Every workflow phase is tagged with one of three behaviors: | Tag | Meaning | What Claude Does | |-----|-----|-----| | [STOP] | Judgment phase — needs your input/approval | Pauses and waits for you | | [AUTO] | Mechanical phase — output is deterministic | Shows a one-line summary, proceeds immediately | | [DELEGATE] | Hands off to

GSD/Superpowers | Shows what was delegated, proceeds | No tag defaults to [STOP] (safe default). AUTO means "don't wait for permission," not "skip" — if an AUTO phase hits something unexpected, Claude will still pause and ask.

PM Mode: GSD + Superpowers Integration Model

All projects use **GSD + Superpowers** together. They have distinct roles: **GSD = orchestration engine** (what to do, when, and tracking progress):

- Project initialization: `/gsd:new-project`
- Phase planning: `/gsd:plan-phase` , `/gsd:discuss-phase`
- Phase execution: `/gsd:execute-phase`
- Verification: `/gsd:verify-work`
- State tracking: `.planning/` directory (STATE.md, ROADMAP.md, phase plans)
- Quick tasks: `/gsd:quick` , `/gsd:fast`
- Test coverage: `/gsd:add-tests`

Superpowers = discipline layer (how to do it well):

- Brainstorming: `superpowers:brainstorming` (before creative or architectural work)
- Git isolation: `superpowers:using-git-worktrees` (feature branches in worktrees)
- TDD enforcement: `superpowers:test-driven-development` (failing test, implement, refactor)
- Code review: `superpowers:requesting-code-review` , `superpowers:receiving-code-review`
- Plan quality: `superpowers:writing-plans` (for non-GSD contexts only, since GSD has its own planning)
- Branch completion: `superpowers:finishing-a-development-branch` (merge or PR)
- Verification discipline: `superpowers:verification-before-completion`

The build loop (how they work together):

- Brainstorm (Superpowers) -> feeds into GSD planning
- Worktree (Superpowers) -> isolation for GSD execution
- Plan (GSD) -> structured task breakdown
- Execute (GSD) -> TDD discipline from Superpowers applies
- Review (Superpowers) -> after GSD execution completes
- Verify (GSD) -> UAT against acceptance criteria
- Finish branch (Superpowers) -> merge or PR

Proactive Toolkit Usage

Claude suggests the next skill automatically after completing each one: | Just finished | What Claude suggests next | |----
-----|-----| | `/scott:new-project` (build) | Code review -> Retro -> Log success | |
`/scott:new-feature` (build) | Code review -> Finish branch | | `/scott:debug` (fix applied) | Log success if clean fix,
resume interrupted work | | `/scott:retro` | Update toolkit if improvements found | | `/scott:handoff` | Retro (project
wrap-up) | | `/gsd:execute-phase` | Code review -> Verify work | | `/gsd:verify-work` (passes) | Finish branch -> Log
success | | Code review passes | Finish branch | | Branch merged | Log success, retro if milestone complete | | Session
wrapping up | Pause | Claude also invokes skills proactively when it detects triggers (bugs, satisfaction, errors, session
end) without being asked.